

Plateforme LogiChain

Partie 3 : Infrastructure DevOps, Intégration Continue et Déploiement Automatisé



1. Contexte et enjeux du transfert d'industrialisation

Les deux premières phases du projet LogiChain ont permis de concevoir une architecture applicative robuste : un back-end Node.js/Express N-Tier connecté à MongoDB, et un front-end mobile React Native fonctionnant en mode Offline-First. L'application entre désormais dans sa phase critique d'industrialisation et de transmission à une seconde équipe de développement et d'exploitation.

Dans un contexte d'ingénierie logicielle professionnelle, la qualité du code ne suffit pas : la pérennité du système repose sur la capacité à automatiser les déploiements, à formaliser la collaboration Git et à garantir une infrastructure reproductible et sécurisée. Vous intervenez en tant que DevOps chargé de construire ce socle d'industrialisation.

2. Objectifs du projet

- **Espace de travail collaboratif** : Structurer un espace de code source partagé intégrant un workflow de branching rigoureux et documenté pour encadrer le travail multi-développeurs.
- **Automatisation de l'infrastructure (IaC)** : Concevoir des scripts Ansible modulaires pour provisionner et configurer l'ensemble du serveur de production de manière totalement automatisée.

- **Pipeline d'intégration continue (CI/CD)** : Automatiser l'exécution des tests, la validation de la qualité de code (linting) et le déploiement applicatif à chaque modification validée.
- **Sécurité et reproductibilité** : Garantir une étanchéité stricte des secrets d'environnement et formaliser un guide de reprise d'activité (Runbook) pour l'équipe réceptrice.

3. Périmètre technique global

L'infrastructure DevOps doit s'appuyer sur la pile technologique standard de l'industrie :

- **Gestion de versions** : GitHub / GitLab avec règles de protection de branches (Branch Protection Rules).
- **Automation & Provisionnement** : Ansible (Playbooks, Roles, Inventory), SSH, Fail2Ban, Nginx.
- **Intégration Continue** : GitHub Actions ou GitLab CI/CD.
- **Gestion de processus & Runtime** : PM2 / Docker pour l'exécution Node.js, instances MongoDB configurées avec sécurité et authentification

4. Spécifications fonctionnelles et livrables DevOps

1. Espace partagé Git et Gitflow documenté

- **Structure de repositories** : Organisation propre des sous-projets (mono-repo ou multi-repos argumentés pour le back end API et le front-end mobile).
- **Stratégie de branching (Gitflow)** : Formalisation d'une convention stricte (main , develop , feature). Documentation explicite des règles de Pull Request / Merge Request, de la revue de code obligatoire et de la politique de commit (Standard Conventional Commits).
- **Documentation de passation (README & Contributing)** : Rédaction d'un guide d'installation en local, des variables d'environnement requises et des instructions de contribution destinées à l'équipe suivante.

2. Automation et Déploiement Serveur via Ansible

- **Rôles Ansible modularisés** : Écriture de Playbooks et Rôles Ansible organisés pour assurer la configuration d'un serveur cible :
 - **Rôle System & Security** : Mise à jour des paquets, création des utilisateurs de service non-root, configuration du pare-feu (UFW) et sécurisation SSH.
 - **Rôle Database** : Installation et sécurisation de MongoDB (création des utilisateurs applicatifs, gestion des accès et indexation initiale).
 - **Rôle Web & Proxy** : Installation et configuration de Nginx avec gestion des certificats SSL/TLS (si possible).
 - **Rôle Application Runtime** : Déploiement de l'API Node.js, installation des dépendances de production et supervision du processus via PM2 ou conteneurs

3. Pipelines CI/CD

- **Validations automatisées** : Déclenchement automatique des pipelines à chaque Push ou Pull Request sur develop et main (exécution du linter, exécution des suites de tests unitaires et d'intégration).
- **Déploiement continu (CD)** : Déploiement automatique ou sur validation sur le serveur de recette/production via l'exécution des Playbooks Ansible.

5. Contraintes et exigences non-fonctionnelles

- **Héritage architectural et défensif** : Les scripts de déploiement doivent respecter les exigences de l'API (architecture N Tier, variables d'environnement strictes). Aucune clé d'API ou secret ne doit être commité en clair dans le dépôt Git.
- **Idempotence des scripts Ansible** : Un Playbook doit pouvoir être exécuté une ou dix fois de suite sur le serveur sans altérer l'état final ni provoquer d'erreur. Les scripts procéduraux ou l'enchaînement de commandes shell brutes (command / shell sans gardes) sont strictement proscrits au profit des modules Ansible natifs.
- **Reproductibilité intégrale** : Le serveur de production doit pouvoir être intégralement reconstruit et l'application redéployée en une seule commande depuis une machine vierge.
- **Documentation d'exploitation (Runbook)** : Fourniture d'un document technique synthétique expliquant la procédure de déploiement, les commandes de rollback et la gestion des sauvegardes de la base de données MongoDB